

Maze Runner Chat History

Visible user and assistant messages from this Copilot session. Internal reasoning and tool-call payloads are omitted.

User

2026-10-09 09:38

Here is my pygame maze game (game_engine.py, maze.py, player.py below). It is supposed to generate a maze, let the player move with WASD/arrows, reach the exit, and regenerate on R. Find every bug that stops this from working, explain each one briefly, and give me the corrected code.

GitHub Copilot

2026-10-09 09:38

The focused movement check passes: a closed east wall blocks the player, and opening that same passage allows crossing. I'm doing a final syntax check and a quick connectivity check on generated mazes to make sure the exit remains reachable after the collision fix. The gameplay-stopping bug was in `Player._hits_wall`: it rejected movement outside the grid but never checked the maze's wall data. That let the player pass through internal walls. The generator and renderer use matching wall directions, and generated mazes are connected, so I found no other blockers in the described flow.

I fixed game/player.py. Here is the corrected file:

```
import pygame
from game.maze import CELL

SPEED = 3

class Player:
    def __init__(self, r, c):
        self.r = r
        self.c = c
        x = c * CELL + CELL // 2
        y = r * CELL + CELL // 2
        self.rect = pygame.Rect(x - 10, y - 10, 20, 20)
        self.color = (60, 120, 220)

    def move(self, keys, walls, rows, cols):
        dx, dy = 0, 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            dx = SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]:
            dy = -SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]:
            dy = SPEED

        new_rect = self.rect.move(dx, 0)
```

```

        if not self._hits_wall(new_rect, walls, rows, cols):
            self.rect = new_rect

        new_rect = self.rect.move(0, dy)
        if not self._hits_wall(new_rect, walls, rows, cols):
            self.rect = new_rect

    def _hits_wall(self, rect, walls, rows, cols):
        for px, py in [
            (rect.left, rect.top),
            (rect.right - 1, rect.top),
            (rect.left, rect.bottom - 1),
            (rect.right - 1, rect.bottom - 1),
        ]:
            cr = py // CELL
            cc = px // CELL
            if cr < 0 or cr >= rows or cc < 0 or cc >= cols:
                return True

        top_cell = rect.top // CELL
        bottom_cell = (rect.bottom - 1) // CELL
        left_cell = rect.left // CELL
        right_cell = (rect.right - 1) // CELL

        for col in range(left_cell, right_cell):
            for row in range(top_cell, bottom_cell + 1):
                if walls[row][col][2]:
                    return True

        for row in range(top_cell, bottom_cell):
            for col in range(left_cell, right_cell + 1):
                if walls[row][col][1]:
                    return True

        return False

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)

```

Verified that movement stops at a closed wall, crosses an open passage, the project compiles, and a generated maze has a path to the exit.

User

2026-10-09 09:44

Add a BFS solver to maze.py that returns the shortest path (list of cells) from the player's current cell to the exit, respecting walls. In game_engine.py, toggle showing it when H is pressed, drawing a semi-transparent colored square on each path cell. Recompute the path when the player moves. Return only changed code.

```

--- game/maze.py
+++ game/maze.py
@@
import random
+from collections import deque
@@
    return walls

+def find_shortest_path(walls, start, goal):
+    rows = len(walls)
+    cols = len(walls[0]) if rows else 0
+    if not rows or not cols:
+        return []
+    if any(not (0 <= r < rows and 0 <= c < cols) for r, c in (start, goal)):
+        return []
+
+    directions = [(-1, 0, 0), (1, 0, 1), (0, 1, 2), (0, -1, 3)]
+    previous = {start: None}
+    queue = deque([start])
+
+    while queue:
+        r, c = queue.popleft()
+        if (r, c) == goal:
+            break
+
+        for dr, dc, wall_dir in directions:
+            nr, nc = r + dr, c + dc
+            neighbor = (nr, nc)
+            if (
+                0 <= nr < rows
+                and 0 <= nc < cols
+                and not walls[r][c][wall_dir]
+                and neighbor not in previous
+            ):
+                previous[neighbor] = (r, c)
+                queue.append(neighbor)
+
+    if goal not in previous:
+        return []
+
+    path = []
+    cell = goal
+    while cell is not None:
+        path.append(cell)
+        cell = previous[cell]
+    return list(reversed(path))
+
+def cell_rect(r, c, import_pygame=None):

```

```

--- game/game_engine.py
+++ game/game_engine.py
@@
-from game.maze import generate_maze, CELL
+from game.maze import generate_maze, find_shortest_path, CELL
@@
    self.font = pygame.font.SysFont("monospace", 22)

```

```

        self.big_font = pygame.font.SysFont("monospace", 36, bold=True)
+       self.show_path = False
        self.reset()

@@
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
+       self.path = find_shortest_path(self.walls, (0, 0), (ROWS - 1, COLS - 1))
        self.exit_rect = pygame.Rect((COLS-1)*CELL+5, (ROWS-1)*CELL+5, CELL-10, CELL-10)

@@
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
+       elif event.type == pygame.KEYDOWN and event.key == pygame.K_h:
+       self.show_path = not self.show_path
@@
        if self.won:
            return
+       previous_cell = (
+           self.player.rect.centery // CELL,
+           self.player.rect.centerx // CELL,
+       )
        keys = pygame.key.get_pressed()
        self.player.move(keys, self.walls, ROWS, COLS)
+       current_cell = (
+           self.player.rect.centery // CELL,
+           self.player.rect.centerx // CELL,
+       )
+       if current_cell != previous_cell:
+           self.path = find_shortest_path(
+               self.walls, current_cell, (ROWS - 1, COLS - 1)
+           )
        self.elapsed = time.time() - self.start_time

@@
    def draw(self):
        self.screen.fill(BG)
+       if self.show_path:
+           path_square = pygame.Surface((CELL, CELL), pygame.SRCALPHA)
+           path_square.fill((30, 170, 220, 90))
+           for r, c in self.path:
+               self.screen.blit(path_square, (c * CELL, r * CELL))
        self.draw_maze()

```

User

2026-10-09 09:47

Add fog of war. Each frame, after drawing the maze and player, draw a black

SRCALPHA overlay and cut out a transparent circle with a radius of 3 cells

(3 * CELL_SIZE) centered on the player. Keep the HUD/timer outside the fog.

It should work with the H hint toggle.

GitHub Copilot

2026-10-09 09:47

The headless rendering check passes: distant playfield pixels are black, the player and nearby H path remain visible, and the HUD pixels are unchanged. I'm checking diagnostics for the edited module before

wrapping up. Added the fog overlay to game/game_engine.py. It covers only the playfield, clears a transparent circle of radius $3 * \text{CELL}$ around the player, and preserves the H hint path within that circle. The HUD remains unfogged.

Verified with a headless pixel check; no diagnostics found.

User

2026-10-09 09:49

Start a timer when the maze begins and stop it on reaching the exit. Save the time to leaderboard.json (top 5, ascending), creating the file if missing and handling a corrupt file. Show the leaderboard on the win screen and highlight the player's new time if it made the list. Pressing R should start a new run.

GitHub Copilot

2026-10-09 09:49

The adjusted stopwatch passes the same workflow check. I'm doing the final compile and diagnostics pass now; the test wrote only to a temporary directory, not the project's leaderboard file. Implemented in game/game_engine.py. The timer starts after maze setup and freezes on reaching the exit. Completion times are saved to leaderboard.json, sorted ascending and limited to five; missing or corrupt data is handled. The win screen shows the leaderboard and highlights the new entry when it qualifies. Pressing R starts a fresh run.

Verified corrupt-file recovery, ranking, win-time freeze, win-screen drawing, compilation, and diagnostics.

User

2026-10-09 09:52

Add a difficulty select screen at startup with clickable Easy (10x8), Medium (15x13) and Hard (20x18) buttons. After a click, generate the maze with those COLS/ROWS and scale CELL_SIZE so it fits the window. R regenerates at the same difficulty. Keep a separate leaderboard per difficulty.

GitHub Copilot

2026-10-09 09:52

Added the startup difficulty menu and fit-to-window maze sizing in game/game_engine.py. Player movement and wall collisions now use the selected cell size and centered maze origin in game/player.py. R regenerates at the current difficulty, and leaderboard.json now keeps separate Easy, Medium, and Hard

boards; existing flat-list scores are retained as Medium.

Verified selection, rendering, viewport fit, scaled collision, R behavior, and leaderboard isolation.
Compilation and diagnostics pass.

User

2026-10-09 09:58

run the game

GitHub Copilot

2026-10-09 09:58

The game is running. Click Easy, Medium, or Hard in the Pygame window to start.

User

2026-10-09 10:00

[Terminal b41a9e57-aa90-4269-8258-ed326bb8c7a0 notification: command completed. The terminal has been cleaned up.]

Terminal output:

PS C:\Users\admin\12_maze_runner> python main.py

C:\Users\admin\AppData\Roaming\Python\Python313\site-packages\pygame\pkgdata.py:25:

UserWarning: pkg_resources is deprecated as an API. See

https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

from pkg_resources import resource_stream, resource_exists

pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)

Hello from the pygame community. <https://www.pygame.org/contribute.html>

User

2026-10-09 10:00

run the game

GitHub Copilot

2026-10-09 10:00

The game is running with system Python. The workspace venv doesn't have Pygame installed, so I used the system interpreter, where Pygame is available.

User

2026-10-09 10:02

[Terminal e4de1d03-e504-4222-a596-0291f5781198 notification: command completed. The terminal has been cleaned up.]

Terminal output:

PS C:\Users\admin\12_maze_runner> python main.py

C:\Users\admin\AppData\Roaming\Python\Python313\site-packages\pygame\pkgdata.py:25:

UserWarning: pkg_resources is deprecated as an API. See

https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

from pkg_resources import resource_stream, resource_exists

pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)

Hello from the pygame community. <https://www.pygame.org/contribute.html>

GitHub Copilot

2026-10-09 10:02

The game is now running as a detached process (PID 19240), so it should stay open after the terminal exits.

User

2026-10-09 10:04

Chat history exported as a pdf

GitHub Copilot

2026-10-09 10:04

I've reconstructed 10 turns from the original session file, including full assistant code blocks. I'll export the visible user/assistant conversation and omit internal reasoning and tool-call payloads, then render the document to `chat_history.pdf`.

Exported from the saved VS Code chat session.